F20BD/F21BD – Big Data Management

CLASS TEST STUDY NOTES

Lectures Covered:

Part 1: Introduction to Big Data (Recap)

Part 2: CAP Theorem & NoSQL Databases

Topics on Class Test:
Intro to Big Data • RDF/RDFS • SPARQL • SKOS

NOT included: OWL / Protege

Heriot-Watt University – AY 2025-26

Week 7 Class Test Preparation

✪ **IMPORTANT:** These notes are written for someone with **zero prior knowledge**. Every concept is explained from scratch with real-world examples.

Last updated: February 24, 2026

Contents

1 Introduction to Big Data – The Big Picture

★ CLASS TEST ALERT:

This Topic IS on the Class Test The Introduction to Big Data is **explicitly listed** as a class test topic. Expect questions on:

- The 5 Vs of Big Data (definitions and examples)
- What makes data "big" – it's relative, not absolute
- Scalability concepts (vertical vs horizontal)
- Data size units (KB, MB, GB, TB, PB, EB, ZB, YB)
- Key historical milestones
- Correlation vs Causation

1.1 What Even IS Big Data? (The Simple Answer)

✤ Real-World Analogy:

Think of a Library Imagine a small town library with 1,000 books. One librarian can manage it easily – find any book, keep track of who borrowed what, and keep things organised.
Now imagine the library suddenly has **1 billion books**, in **50 different languages**, with **new books arriving every second**, and some books have **missing pages or wrong covers**. That one librarian can't cope anymore. You need a whole new system.
That's Big Data – when your data gets so large, so varied, and arrives so fast that your normal tools can't handle it anymore.

✦ Definition:

Big Data Big Data is an **ambiguous and relative concept**. It refers to datasets that are so large, complex, and fast-growing that traditional software (like Excel or a normal database) **cannot process them** effectively.

Key point: What counts as "big" changes over time. In 1880, processing census data for the entire US was "big data". Today, that same data would fit on a USB stick.

1.2 A Brief History of Big Data

✪ EXAM TIP:

Historical dates are often tested You don't need to memorise every date, but know the **key milestones** and what they represent. Focus on the ones marked with a star (★).

1663	★ John Graunt: first statistical data analysis (bubonic plague)
1880	US Census Bureau: estimated 8 years to process data
1880	★ Hollerith creates tabulating machine (3 months instead of 8 years!)
1927	Fritz Pfleumer invents magnetic tape data storage
1945	Von Neumann creates EDVAC computer
1989	★ Tim Berners-Lee creates the World Wide Web (WWW)
1999	IoT (Internet of Things) invented
2005	★ Roger Magoulas coins the term "Big Data"
2005	★ Hadoop created (based on Nutch + Google MapReduce)
2013	IoT becomes generalised across all sectors

★ Example:

Why Hollerith's Machine Matters In 1880, the US Census took an estimated **8 years** to process manually. Herman Hollerith built a **tabulating machine** that processed the same data in just **3 months**. This is one of the first examples of using technology to handle "big data" – and the company Hollerith founded eventually became **IBM**.

1.3 The Era of Data – Why Now?

In 2013, a staggering statistic emerged: **90% of the world's data was created in just the previous 2 years** (source: SINTEF). Three driving forces explain this:

1. **Mobile Sensors and Trackers** – your phone, smartwatch, fitness band all generate data constantly
2. **IoT (Internet of Things)** – smart fridges, connected cars, factory sensors, hospital monitors
3. **Systematic Data Collection** – companies collect data on everything you do online

1.3.1 How Much Data Are We Talking About?

✪ EXAM TIP:

Data Units – KNOW THIS TABLE This table is **very likely to appear** on the exam. You need to know the conversion factor (each is 1000× the previous).

Value	Abbrev.	Full Name	Real-World Example
1000^1	kB	Kilobyte	A short email
1000^2	MB	Megabyte	One MP3 song (≈3–5 MB)
1000^3	GB	Gigabyte	A movie in HD (≈4 GB)
1000^4	TB	Terabyte	A large external hard drive
1000^5	PB	Petabyte	All of Netflix's content
1000^6	EB	Exabyte	All words ever spoken by humans
1000^7	ZB	Zettabyte	Total internet traffic per year
1000^8	YB	Yottabyte	Theoretical – doesn't exist yet

★ Example:

Putting It in Perspective The world will produce approximately:

- **2030:** 2,537 Zettabytes of data
- **2035:** 19,267 Zettabytes of data

That's like filling **trillions** of external hard drives every single year.

1.3.2 Data from Engineering Sources

A city of one million people will generate **200 million gigabytes of data per day** by 2020 (Cisco estimate). Here's where it comes from:

Source	Data/Day	% Transmitted
Connected Plane	40 TB	0.1%
Connected Factory	1 PB	0.2%
Public Safety	50 PB	<0.1%
Weather Sensors	10 MB	5%
Intelligent Building	275 GB	1%
Smart Hospital	5 TB	0.1%
Smart Car	70 GB	0.1%
Smart Grid	5 GB	1%

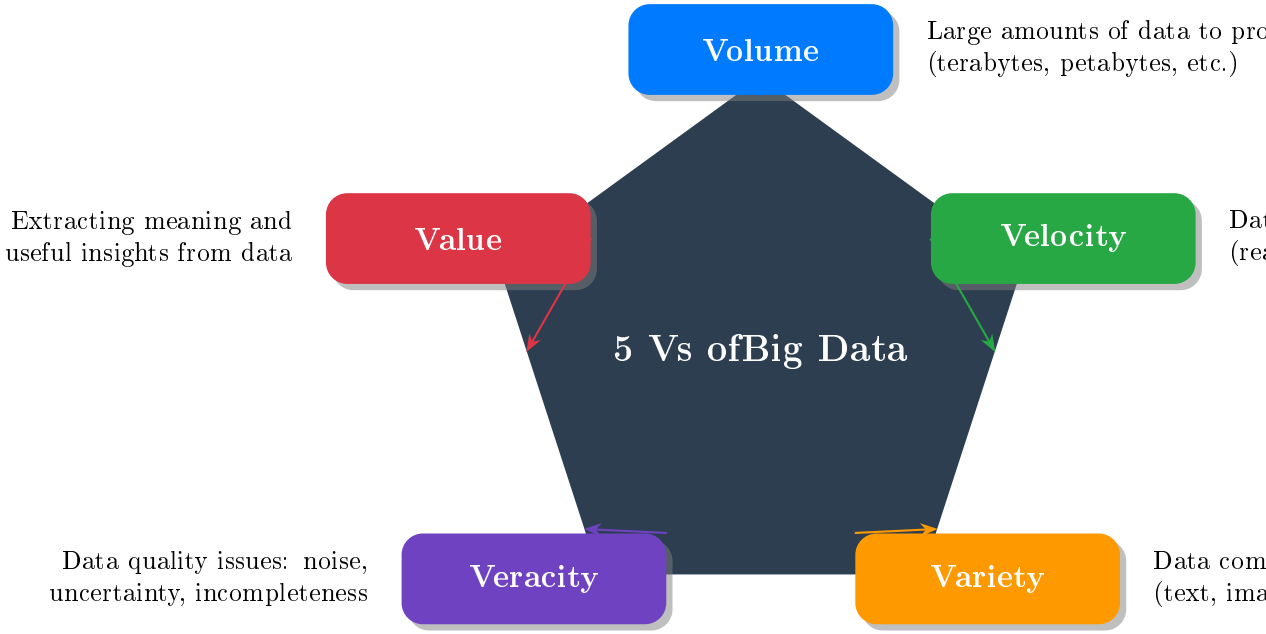
❖ Key Concept:

Most Data is Never Transmitted Notice that most sources only transmit a **tiny fraction** of the data they generate (often less than 1%). This is a key insight – we generate far more data than we can ever send over the internet.

1.4 The 5 Vs of Big Data

★ CLASS TEST ALERT:

The 5 Vs – Almost Guaranteed on Test The 5 Vs of Big Data are **the most fundamental concept** in this course. You MUST be able to name all 5, define each, and give examples. This is practically guaranteed to appear on the class test.



Let's break each V down with simple explanations:

✦ Definition:

Volume **What it means:** The sheer *amount* of data is enormous.
Think of it like this: Imagine trying to count every grain of sand on a beach. That's what dealing with petabytes of data feels like.
Examples: Facebook stores over 300 petabytes of user data. YouTube gets 500 hours of video uploaded every minute.

✦ Definition:

Velocity **What it means:** Data arrives and accumulates at very *high speed*.
Think of it like this: Imagine a fire hose spraying water at you – you need to catch and sort every droplet in real time. That's velocity.
Examples: Stock market data changes every millisecond. Twitter generates thousands of tweets per second. IoT sensors send readings continuously.

✦ Definition:

Variety **What it means:** Data comes in many different *formats* and types.
Think of it like this: Imagine receiving letters, phone calls, text messages, photos, videos, and voice notes all at once – they're all "messages" but in totally different formats.
Three types of data:

- **Structured:** Neat rows and columns (like an Excel spreadsheet)
- **Semi-structured:** Has some organisation but is flexible (like JSON or XML)
- **Unstructured:** No fixed format (like emails, videos, social media posts)

✦ Definition:

Veracity **What it means:** How *trustworthy* and *reliable* is the data?
Think of it like this: If 100 people tell you the weather forecast, but 30 of them are guessing, 10 are lying, and 5 are reading yesterday's report – that's a veracity problem.
Issues include: Consistency, noise (random errors), uncertainty, completeness (missing data),

timeliness (is it up to date?).

✦ Definition:

Value **What it means:** Can we extract *useful insights* from all this data?
Think of it like this: Having a mountain of gold ore is useless until you mine it and extract the actual gold. Data is the ore; value is the gold.
Examples: Netflix uses viewing data to recommend shows. Hospitals use patient data to predict disease outbreaks.

1.5 Data-Driven Research

Modern research follows four steps:

1. **Experiment** – design and run an experiment
2. **Acquire data** – collect the results
3. **Analyse data** – look for patterns and insights
4. **Elaborate models/theories** – build a theory that explains what you found

This applies to fields like genomics (studying genes), astronomy (astroML), environmental science, and medicine.

1.6 Correlation vs Causation – A Critical Warning

☞ Warning:

Correlation ≠ Causation Just because two things happen at the same time does NOT mean one causes the other. This is one of the biggest mistakes in data analysis, and the lectures **strongly emphasise** this.

✤ Real-World Analogy:

The Ice Cream and Drowning Problem **Fact:** Ice cream sales and drowning deaths both increase in summer.
Wrong conclusion: "Ice cream causes drowning!"
Right conclusion: Both are caused by a **third factor** – hot weather. People swim more AND eat more ice cream when it's hot. The ice cream has nothing to do with the drowning.

★ Example:

Funny Spurious Correlations from the Lecture The lecture shows real examples from `tylervigen.com/spurious-correlations`:

- Number of Nicolas Cage films correlates with pool drownings (r=0.666)
- Computer science doctorates correlate with arcade revenue (r=0.985)
- Cheese consumption correlates with bedsheet tangling deaths (r=0.947)

These are **all nonsense** – high correlation but absolutely no causal link!

✪ EXAM TIP:

Google Flu Trends – A Famous Big Data Example Google tried to predict flu outbreaks by analysing search queries. Initially, it worked well, but eventually **diverged significantly** from real CDC data (overestimating by 4.3 percentage points). This demonstrates that **big data**

analysis can go wrong if you confuse correlation with causation or don't account for changing user behaviour.

1.7 Big Data Challenges and Scalability

❖ Key Concept:

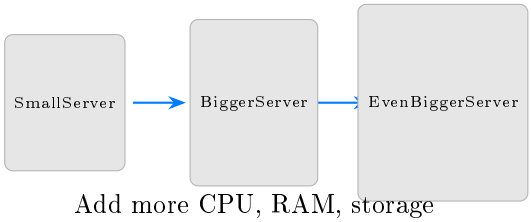
The Core Challenge Big Data requires:

- **Increased scalability** – in storage, querying, and processing
- **Fast analytics** – getting answers quickly from massive datasets

The question is: *How do we scale up?*

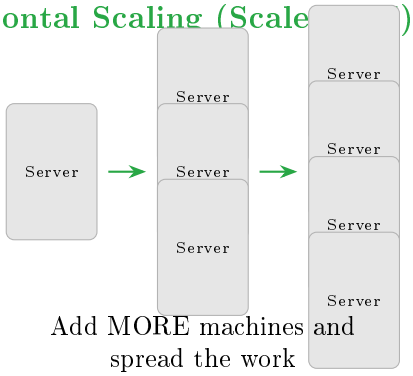
1.7.1 Two Types of Scalability

Vertical Scaling (Scale UP)



Add more CPU, RAM, storage to ONE machine
(Expensive, has limits!)

Horizontal Scaling (Scale)



Add MORE machines and spread the work
(Cheaper, more flexible!)

Feature	Vertical (Scale Up)	Horizontal (Scale Out)
What you do	Add more power to ONE machine	Add MORE machines
Example	Upgrade RAM from 16GB to 128GB	Add 10 more servers
Cost	Expensive (high-end hardware)	Cheaper (commodity hardware)
Limit	Has a ceiling (can't upgrade forever)	Almost unlimited
Complexity	Simple (just one machine)	Complex (distributed computing)
Used by	Traditional RDBMS	Big Data / NoSQL systems

2 CAP Theorem and NoSQL Databases

★ CLASS TEST ALERT:

CAP & NoSQL – Likely on the Exam While the class test announcement lists "Introduction to Big Data" explicitly, the CAP theorem and NoSQL content builds directly on the scalability concepts from the intro lectures. Understanding distributed databases, ACID vs BASE, and the CAP theorem is **fundamental background knowledge** for the course. Understand these concepts thoroughly.

2.1 Traditional Relational Databases (RDBMS) – Quick Recap

✤ Real-World Analogy:

Think of a Spreadsheet A **relational database** is like a very powerful, very organised spreadsheet. Data is stored in **tables** (like Excel sheets) with **rows** (individual records) and **columns** (attributes/fields).
Everything is structured, connected through relationships, and follows strict rules to keep data consistent.

✦ Definition:

Relational Database (RDBMS) A database system that stores data in structured tables with predefined schemas (column names, data types). Uses SQL (Structured Query Language) for queries. Invented in the 1970s.
Key strengths: High consistency, well-founded theory (relational algebra), excellent for centralised storage.
Examples: Oracle, MySQL, PostgreSQL, Microsoft SQL Server.

2.2 ACID Properties – The Rules of Traditional Databases

✪ EXAM TIP:

ACID vs BASE Comparison Knowing the difference between ACID and BASE is a **very common exam question**. Memorise what each letter stands for and be able to explain the difference.

ACID is a set of four guarantees that traditional (relational) databases follow to ensure data is always reliable:

A **Atomicity**

All parts of a transaction succeed, or NONE of them do. It's all-or-nothing.
Like transferring money: either BOTH the debit AND credit happen, or neither does.

C **Consistency**

Data must be valid before AND after every transaction. All rules are followed.
Like a bank: the total money in all accounts must always add up correctly.

I **Isolation**

Multiple transactions happening at the same time don't interfere with each other.
Like two people using ATMs simultaneously – each transaction is independent.

D **Durability**

Once a transaction is complete, the changes are permanent – even if the system crashes.
Like writing in permanent ink – once it's done, it stays done.

2.3 Why Traditional Databases Struggle with Big Data

❖ Key Concept:

The Problem Traditional relational databases are great for **centralised storage** (one big server). But when you need to handle massive internet-scale data across many servers, they hit serious problems:

- **Slow response times** with massive amounts of data
- **Expensive to scale up** (need bigger and bigger hardware)
- **Limited analytical capabilities** (need special schemas like OLAP)
- **Distribution is hard** – keeping all copies in sync is extremely difficult

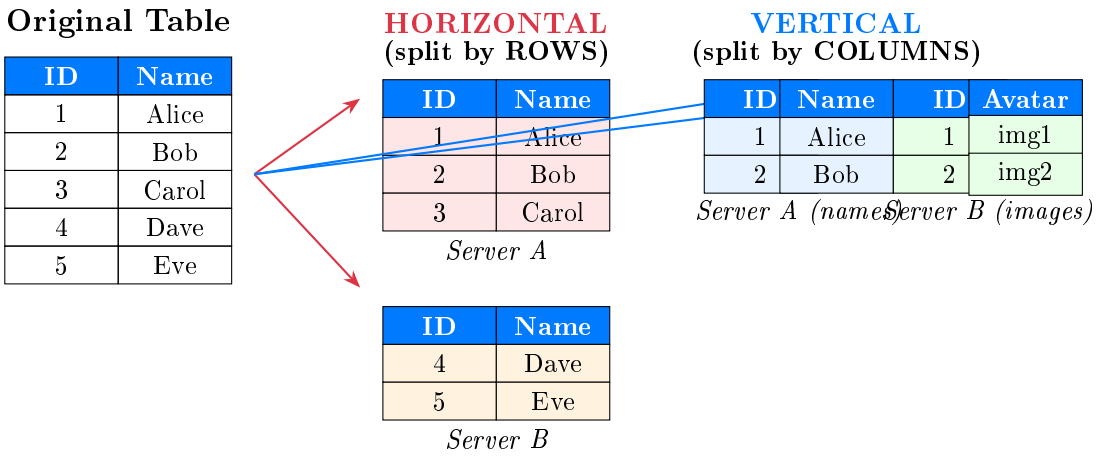
2.4 Distributed Databases – Spreading Data Across Machines

✦ Definition:

Distributed Database A database where data is stored across **multiple computers** (called nodes) that are connected by a network. The system manages everything so the user doesn't know or care which computer their data is on.

2.4.1 Fragmentation (Partitioning)

When you split a table across multiple machines, it's called **fragmentation** or **partitioning**:



✦ Definition:

Database Sharding **Sharding** is just another word for partitioning, but specifically means the fragments (**shards**) are stored on **different computers**.
Real example: In 2011, Facebook split its MySQL database into **4,000 shards** to handle its massive data volume.

2.4.2 Replication

Replication means keeping **copies** of data on multiple servers. If one server goes down, another has the same data. This improves **availability** but creates a problem: how do you keep all copies in sync?

2.5 Properties of Distributed Systems (C, A, P)

Before we understand the CAP theorem, we need to understand its three properties:

Property	Definition	Simple Explanation
Consistency (C)	Every read returns the most recent write. All nodes see the same data at the same time.	When you update your profile photo, **everyone** sees the new photo immediately – nobody sees the old one.
Availability (A)	Every request gets a response (not an error), even if some nodes are down.	The website **always responds** to you, even if some servers have crashed. You always get an answer.
Partition Tolerance (P)	The system keeps working even if some messages between nodes get lost (network problems).	Even if the cable between two data centres gets cut, the system **doesn't completely crash**.

2.6 The CAP Theorem

★ CLASS TEST ALERT:

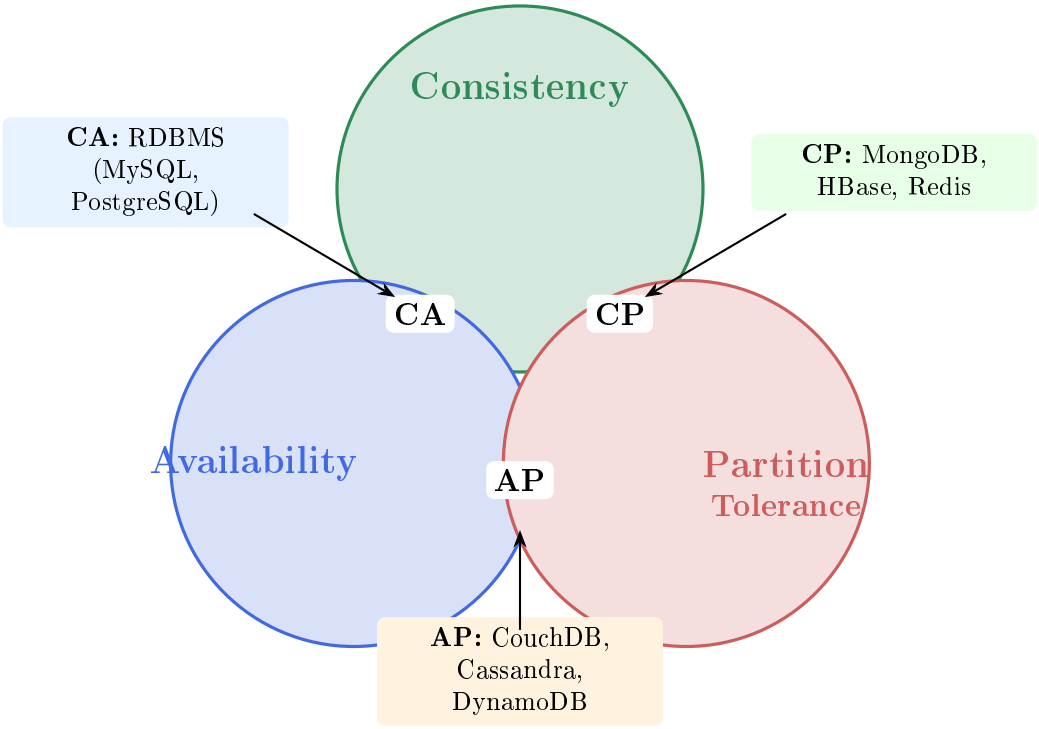
CAP Theorem – Core Concept The CAP theorem is one of the **most important concepts** in distributed computing and Big Data. Understand it deeply.

✦ Definition:

The CAP Theorem A distributed system can guarantee **at most TWO** of the three properties (Consistency, Availability, Partition Tolerance) **at the same time**. You **cannot have all three**.

History:

- **2000:** Eric Brewer proposed this as a **conjecture** (an educated guess)
- **2002:** Seth Gilbert and Nancy Lynch **proved it as a theorem** (mathematically true)



2.6.1 Understanding Each Combination

❖ **Key Concept:**

CA – Consistency + Availability **What you get:** Consistent and always available.
What you sacrifice: Cannot handle network partitions (if the network splits, the system breaks).
Who uses it: Traditional RDBMS (MySQL, PostgreSQL, Oracle).
Real-world analogy: A single office with one filing cabinet. Everyone in the office sees the same files (consistent) and can always access them (available). But if the office floods (partition), everything stops.

❖ **Key Concept:**

CP – Consistency + Partition Tolerance **What you get:** Data is always correct, even during network problems.
What you sacrifice: Availability – the system might refuse to answer rather than give wrong data.
Who uses it: MongoDB, HBase, Redis.
How it works: Uses quorum/majority voting (e.g., Paxos protocol). Nodes must agree before responding.
Real-world analogy: A bank that would rather say "sorry, system unavailable" than accidentally show you a wrong balance.

❖ **Key Concept:**

AP – Availability + Partition Tolerance **What you get:** System always responds, even during network problems.
What you sacrifice: Consistency – you might get slightly outdated data.

Who uses it: CouchDB, Cassandra, DynamoDB, Riak.
Uses: Eventual consistency – data will become consistent *eventually*, just not immediately.
Real-world analogy: Social media "likes" counter. You might see 999 likes while your friend sees 1,001, but eventually you'll both see 1,002. It's okay to be slightly wrong temporarily.

✤ Real-World Analogy:

The Pizza Delivery Dilemma Imagine you run a pizza delivery with three promises:

1. **Consistency:** Every customer gets the exact pizza they ordered
2. **Availability:** You ALWAYS deliver (never say "sorry, we're closed")
3. **Partition Tolerance:** You can still deliver even when some roads are blocked

Now imagine a road gets blocked (partition). You have two choices:

- **CP approach:** Wait until the road is clear, then deliver the right pizza (consistent but slow/unavailable)
- **AP approach:** Send whatever pizza is available from the nearest location (available but might be wrong)

You can't promise all three when roads are blocked!

2.7 NoSQL Databases – The Big Data Solution

✦ Definition:

NoSQL **NoSQL** stands for "**Not Only SQL**" (originally "no more SQL"). These are database systems designed for:

- Massive amounts of distributed data
- Flexible schemas (no rigid table structure)
- Horizontal scaling (adding more machines)
- High availability even at the cost of some consistency

2.7.1 Brief History of NoSQL

Year	Event
1998	Carlo Strozzi uses "noSQL" to name a database queried without SQL
2000	Neo4j (graph database) created
2004	Google BigTable (wide-column) created
2005	CouchDB created
2007	Amazon DynamoDB research paper published
2008	Cassandra open-sourced by Facebook
2009	"NoSQL" used as the name of a conference (Rackspace)

2.8 ACID vs BASE – The Key Comparison

✪ EXAM TIP:
ACID vs BASE – High Priority for Exam If you only memorise one comparison table, make it this one. The contrast between ACID and BASE is foundational to understanding why NoSQL exists.

	ACID (Relational / SQL)	BASE (NoSQL)
Used by	Traditional RDBMS	NoSQL databases
Consistency	**Strong** – always correct	**Weak/Eventual** – correct later
Availability	Less available	**Availability first**
Focus	Transactions, "commit"	Best effort, approximate
Approach	Conservative (pessimistic)	Aggressive (optimistic)
Schema	Difficult to change	Easy to evolve
Speed	Slower (more guarantees)	Faster (fewer guarantees)
Scaling	Vertical (scale up)	Horizontal (scale out)

✦ Definition:
BASE Principles (Expanded) **B**asic **A**vailability – The database is always up and running thanks to replication across many machines. Even if some machines fail, the system stays available.
Soft State – The data might not be consistent at all times. Consistency is the *developer's problem*, not the database's problem.
Eventual Consistency – If you stop writing new data and wait, all copies will *eventually* converge to the same value. Like how gossip eventually reaches everyone in a school.

2.9 Four NoSQL Data Models

✪ EXAM TIP:
Know the 4 Types Be able to name the four NoSQL data models, describe how each stores data, and give at least one example database for each.

2.9.1 1. Key-Value Store

How it works: Like a dictionary – each piece of data has a unique **key** and a **value**

The value is "opaque": The database doesn't know or care what's inside the value. It just stores and retrieves it by key.

Schema-less: No fixed structure

Mostly in-memory: Very fast because data is in RAM

Examples: Redis, Riak KV, Oracle NoSQL, Amazon DynamoDB, Azure Cosmos DB

2.9.2 2. Document Store

- **How it works:** Like key-value, but the value is a **structured document** (usually JSON or XML)
- **Key difference from key-value:** The database CAN look inside the document and query individual fields
- **Documents are indexed by a unique key**

- **Includes search capabilities**
- **Examples:** MongoDB, CouchDB, DynamoDB, Azure Cosmos DB

★ Example:

Document Store – JSON Example

```
{
  "_id": "user_101",
  "name": "Alice",
  "age": 25,
  "hobbies": ["reading", "cycling"],
  "address": {
    "city": "Edinburgh",
    "postcode": "EH1 1AA"
  }
}
```

Unlike a key-value store, the database can search by `name`, `age`, or even `address.city` directly.

2.9.3 3. Wide-Column Store

- **Also called:** Column family databases
- **How it works:** Uses flexible tables where different rows can have **different columns**
- **Sparse data:** Not every row needs to have a value in every column (unlike traditional tables)
- **Vertical sharding:** Column families stored on separate computers
- **Horizontal sharding:** Rows within a column family stored on different computers
- **Examples:** Cassandra, Google BigTable, HBase

★ Example:

Wide-Column – How It Differs from Relational In a relational database, every row MUST have all columns:

Name	Email	Phone	Fax
Alice	alice@x.com	123-456	*NULL*
Bob	bob@y.com	*NULL*	*NULL*

In a wide-column store, each row only stores what it HAS:

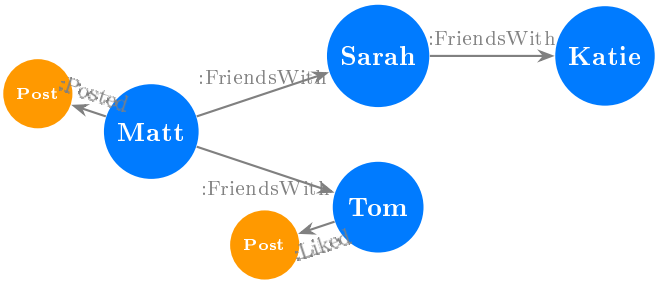
Row Key	Columns (only what exists)
Alice	email:alice@x.com, phone:123-456
Bob	email:bob@y.com

No wasted NULLs! Much more space-efficient for sparse data.

2.9.4 4. Graph Database

- **How it works:** Focuses on **relationships** between entities
- **Nodes** = entities (people, products, locations)
- **Edges** = relationships/connections between entities

- **Perfect for:** Social networks, recommendation engines, fraud detection
- **Examples:** Neo4j, InfiniteGraph



2.10 NoSQL and CAP – Which Goes Where?

Database	Type	CAP	Key Feature
MySQL, PostgreSQL, Oracle	Relational	CA	Strong consistency
MongoDB	Document	CP	Consistent even with partitions
HBase	Wide-column	CP	Consistent + partition tolerant
Redis	Key-value	CP	Fast, in-memory, consistent
CouchDB	Document	AP	Always available
Cassandra	Wide-column	AP	Highly available, eventual consistency
DynamoDB	Key-value/Doc	AP	AWS managed, highly available
Riak	Key-value	AP	Distributed, always available

2.11 PACELC Theorem – The Extension of CAP

✦ **Definition:**

PACELC Theorem The PACELC theorem extends CAP by saying:
If there is a Partition (P): you must choose between **Availability (A)** and **Consistency (C)**.
Else (E), during normal operation: you must choose between **Latency (L)** and **Consistency (C)**.

In short: even when everything is working fine (no partition), you STILL have to choose between being **fast** (low latency) or being **perfectly accurate** (consistent).

Database	On Partition: A or C?	Else: L or C?	PACELC
Cassandra	Availability (PA)	Latency (EL)	PA/EL
MongoDB	Consistency (PC)	Consistency (EC)	PC/EC
DynamoDB	Availability (PA)	Latency (EL)	PA/EL
MySQL Cluster	Consistency (PC)	Consistency (EC)	PC/EC
Riak	Availability (PA)	Latency (EL)	PA/EL

2.12 NoSQL Summary – Characteristics

❖ Key Concept:

NoSQL Key Characteristics (Summary)

- **Distributed** – sharding (splitting data) + replication (copying data)
- **No ACID** – replaced by BASE (eventual consistency)
- **No standard query language** – each system has its own API/query language
- **No predefined schema** – dynamic, flexible structure
- **Horizontal scaling** – add more cheap machines instead of one expensive machine
- **Different flavours** for different use cases (key-value, document, wide-column, graph)

2.13 RDB vs NoSQL – Final Comparison

Feature	Relational (SQL)	NoSQL
Data Model	Fixed tables, rows, columns	Flexible (key-value, document, graph, column)
Schema	Predefined, rigid	Dynamic, flexible
Query Lang.	SQL (standardised)	Various (no standard)
Consistency	Strong (ACID)	Eventual (BASE)
Scaling	Vertical (scale up)	Horizontal (scale out)
Best For	Structured data, transactions	Big data, high availability, flexible data
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis, Neo4j
CAP Model	CA	CP or AP

3 Practice Questions for the Class Test

★ CLASS TEST ALERT:

Test Yourself! Try to answer these **without looking at the notes first**. Then check your answers. These are designed to mimic the style of questions you'll see on the class test.

3.1 Short Answer Questions

Q1. Name the **5 Vs** of Big Data and give a one-sentence definition for each.

Q2. What is the difference between **vertical scaling** and **horizontal scaling**? Which one is preferred for Big Data?

Q3. What does the **CAP theorem** state? Who proposed it and when was it proven?

Q4. A company wants to build a social media platform that must **always respond** to users, even if some servers go down, and can tolerate temporary inconsistencies. Which CAP model should they choose? Name one database that fits.

Q5. Explain the difference between **ACID** and **BASE** properties. Which is used by SQL databases and which by NoSQL?

Q6. What is **database sharding**? Give a real-world example from the lectures.

Q7. Name the **four NoSQL data models** and give one example database for each.

Q8. Why does **correlation not equal causation**? Give one example from the lectures.

Q9. What is the difference between a **key-value store** and a **document store**?

Q10. What does **eventual consistency** mean in the context of NoSQL databases?

3.2 Answers

A1. **Volume** (huge amount of data), **Velocity** (data arrives at high speed), **Variety** (data in many formats), **Veracity** (data quality/trustworthiness), **Value** (extracting useful insights).

A2. **Vertical** = making one machine more powerful (more CPU, RAM). **Horizontal** = adding more machines and distributing the workload. Horizontal is preferred for Big Data because it's cheaper and has no upper limit.

A3. The CAP theorem states that a distributed system can guarantee at most **two of three** properties: Consistency, Availability, and Partition Tolerance. Proposed by **Eric Brewer in 2000** as a conjecture, proven by **Gilbert and Lynch in 2002**.

A4. **AP** (Availability + Partition Tolerance). Examples: Cassandra, CouchDB, DynamoDB, Riak.

A5. **ACID** (Atomicity, Consistency, Isolation, Durability) provides strong consistency and is used by SQL/relational databases. **BASE** (Basic Availability, Soft State, Eventual Consistency) provides high availability with weaker consistency and is used by NoSQL databases.

A6. Sharding is splitting a database table into smaller pieces (shards) stored on different computers. **Facebook** split its MySQL database into 4,000 shards in 2011.

A7. (1) **Key-Value**: Redis, (2) **Document**: MongoDB, (3) **Wide-Column**: Cassandra, (4) **Graph**: Neo4j.

A8. Two things happening together doesn't mean one causes the other. Example: Nicolas Cage films correlate with pool drownings (r=0.666), but Cage's movies obviously don't cause drownings.

A9. In a key-value store, the value is **opaque** (the database can't look inside it). In a document store, the value is a **structured document** (JSON/XML) and the database **can query individual fields** inside it.

A10. Eventual consistency means that after a write/update, all copies of the data will **eventually** become consistent (the same), but there may be a short period where different nodes show different values.

4 Quick Reference Cheat Sheet

CHEAT SHEET – One Page Summary

5 Vs of Big Data

- **V olume** – How much data
- **V elocity** – How fast it arrives
- **V ariety** – How many formats
- **V eracity** – How trustworthy
- **V alue** – How useful

ACID (SQL)

- **A**tomicity – all or nothing
- **C**onsistency – always valid
- **I**solation – transactions independent
- **D**urability – changes permanent

BASE (NoSQL)

- **B**asic **A**vailability – always up
- **S**oft State – may be inconsistent
- **E**ventual Consistency – syncs later

CAP Theorem

- Pick only 2 of 3: C, A, P
- RDBMS = CA
- MongoDB/HBase/Redis = CP
- Cassandra/CouchDB/DynamoDB = AP
- Brewer 2000 (conjecture)
- Gilbert & Lynch 2002 (theorem)

4 NoSQL Models

1. **Key-Value** – dictionary lookup
   Redis, Riak, DynamoDB
2. **Document** – structured JSON/XML
   MongoDB, CouchDB
3. **Wide-Column** – flexible tables
   Cassandra, HBase, BigTable
4. **Graph** – nodes + edges
   Neo4j, InfiniteGraph

Scaling

- **Vertical (Up)** = bigger machine (RDBMS)
- **Horizontal (Out)** = more machines (NoSQL)

Key Dates

- 1663 – First statistical data analysis
- 1880 – Hollerith tabulating machine
- 1989 – WWW created
- 2005 – "Big Data" term coined
- 2005 – Hadoop created

✪ Key Warnings

- Correlation $\neq$ Causation!
- Google Flu Trends failed
- 90% of data created in last 2 years (2013)
- Data units: KB→MB→GB→TB→PB→EB→ZB→YB (each $\times 1000$)

PACELC

- On **P**artition: **A** vs **C**
- **E**lse: **L**atency vs **C**onsistency

★ CLASS TEST REMINDER

Topics included: Introduction to Big Data, RDF/RDFS, SPARQL, SKOS
Topics NOT included: OWL/Protege

These notes cover **Introduction to Big Data**
and foundational **CAP/NoSQL** concepts.
Separate notes will cover RDF/RDFS, SPARQL, and SKOS.